Interpolation - Part 3
Numerical Differentiation

Alexandra A. Gomes

April 19, 2026

Flaws of Polynomial Interpolation

Recall that both polynomial and Hermite interpolation approximate f, in a closed interval, with **one** polynomial.

- the error term in
$$\frac{f^{(n+1)}\left(\xi\left(x\right)\right)}{(n+1)!}$$
may be big;
- high-order polys may oscillate erratically: a small fluctuation in a tiny portion of $[a, b]$ can induce large oscillations over the entire $[a, b]$;

Piecewise Polynomial Interpolation: Basic Idea

Piecewise Polynomial Interpolation: divide the interval into subintervals and construct an approximating polynomial for each subinterval.

Basic Approaches

- **piecewise-linear interpolation**:
  - join points $(x_0, f(x_0)), (x_1, f(x_1)), \ldots (x_n, f(x_n))$ by straight lines;
  - in general, there is no differentiability at the endpoints of the subintervals, which implies the lack of smoothness of the approximating function
- **piecewise Hermite polynomial**:
  - for each subinterval we have 2 points (endpoints); therefore, $n = 1$ and we can construct a cubic Hermite poly, which is smoother than a piecewise linear poly;
  - but we need to know f' at the endpoints

Cubic Spline Interpolation

- cubic polynomials in each subinterval
- 4 constants to be determined for each cubic polynomial: $ax^3 + bx^2 + cx + d$
- use these constants to impose smoothness on the approximating polynomial on the interval

Cubic Spline Interpolant: Definition

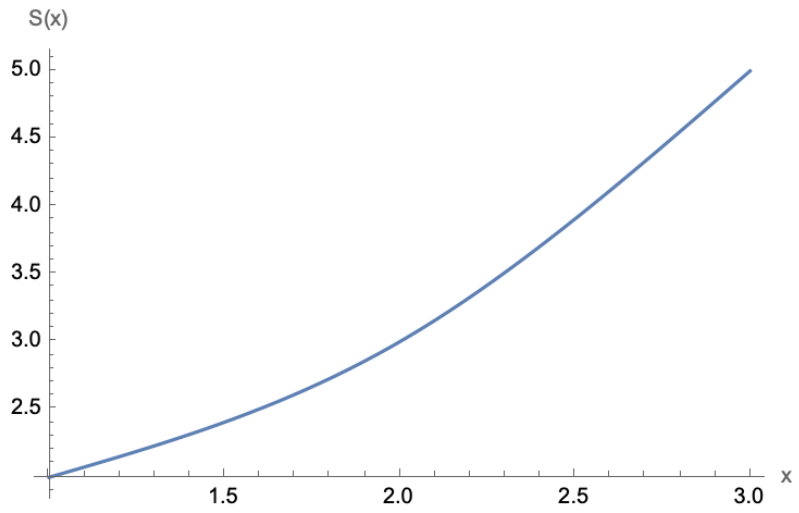
Consider a function f defined on $[a, b]$ and nodes $a = x_0 < x_1 < \ldots < x_n = b$. A **cubic spline interpolant** S for f is a function that satisfy the following conditions:

1. $S(x) = S_j(x)$, a cubic polynomial on $[x_j, x_{j+1}]$, for $j = 0, 1, \ldots, n-1$;
2. $S_j(x_j) = f(x_j)$ and $S_j(x_{j+1}) = f(x_{j+1})$, for $j = 0, 1, \ldots, n-1$, which implies $S_{j+1}(x_{j+1}) = S_j(x_{j+1})$, for $j = 0, 1, \ldots, n-2$;
3. $S'_{j+1}(x_{j+1}) = S'_j(x_{j+1})$, for $j = 0, 1, \ldots, n-2$;
4. $S''_{j+1}(x_{j+1}) = S''_j(x_{j+1})$, for $j = 0, 1, \ldots, n-2$;
5. choose one from two sets of boundary conditions:
   - 5.1 natural or free boundary: $S''(x_0) = S''(x_n) = 0$;
   - 5.2 clamped boundary: $S'(x_0) = f'(x_0)$ and $S'(x_n) = f'(x_n)$;

Cubic Spline Interpolant: Example

Natural cubic spline through $(1,2)$, $(2,3)$ and $(3,5)$?

$$S(x) = \begin{cases} 2 + \frac{3}{4}(x-1) + \frac{1}{4}(x-1)^3, x \in [1,2] \\ 3 + \frac{3}{2}(x-2) + \frac{3}{4}(x-2)^2 - \frac{1}{4}(x-2)^3, x \in [2,3] \end{cases}$$



Constructing the Cubic Spline

$$S_j(x) = a_j + b_j(x - x_j) + c_j(x - x_j)^2 + d_j(x - x_j)^3$$

1. $S_j(x_j) = a_j$
2. $h_j = x_{j+1} - x_j$
3.

$$\begin{aligned} S_{j+1} \quad (x_{j+1}) &= a_{j+1} = S_j(x_{j+1}) = \\ &= a_j + b_j(x_{j+1} - x_j) + c_j(x_{j+1} - x_j)^2 + d_j(x_{j+1} - x_j)^3 \\ &= a_j + b_j h_j + c_j h_j^2 + d_j h_j^3 \end{aligned}$$

4. $S_j'(x_j) = b_j$
5. $b_{j+1} = b_j + 2c_j h_j + 3d_j h_j^2$
6. ...

Constructing the Cubic Spline

$$S_j(x) = a_j + b_j(x - x_j) + c_j(x - x_j)^2 + d_j(x - x_j)^3$$

1. ...
2. $b_{j+1} = b_j + h_j(c_j + c_{j+1})$
3. $c_{j+1} = c_j + 3d_j h_j$

$$h_{j-1}c_{j-1} + 2(h_{j-1} + h_j)c_j + h_j c_{j+1} = \frac{3}{h_j}(a_{j+1} - a_j) - \frac{3}{h_{j-1}}(a_j - a_{j-1})$$

for $j = 1, 2, \ldots, n-1$

Cubic Spline Construction: a System of Linear Equations

$$h_{j-1}c_{j-1}+2\left(h_{j-1}+h_j\right)c_j+h_jc_{j+1}=\frac{3}{h_j}\left(a_{j+1}-a_j\right)-\frac{3}{h_{j-1}}\left(a_j-a_{j-1}\right)$$

for $j=1,2,\ldots,n-1$

1. a_j's are known, h_j are also known; c_j's are the unknowns;
2. can we determine the c_j's? if so, are they unique?

Theorem: Natural Cubic Spline

If $a = x_0 < \ldots < x_n = b$ belong to the domain of f, then f has a unique natural cubic spline interpolant S on the nodes $x_0, \ldots, x_n$.

Theorem: Clamped Cubic Spline

If $a = x_0 < \ldots < x_n = b$ belong to the domain of f and f is differentiable at a and b, then f has a unique clamped cubic spline interpolant S on the nodes $x_0, \ldots, x_n$.

Smoothness

In general, splines are better in data fitting than ordinary polynomials. Why?

1. for the cubic spline, we have continuity of S, S' and S''; the graph appears smooth to the eye (discontinuities in S''' cannot be detected visually);
2. recall that interpolation by ordinary polynomials may exhibit wild oscillations:

   Cubic Spline Smoothness Theorem: for $f \in C^2[a, b]$, $a = x_0 < \ldots < x_n = b$ in the domain of f:

$$\int_a^b \left[S''(x)\right]^2 dx \leq \int_a^b \left[f''(x)\right]^2 dx$$

Numerical Differentiation: Goal

- we want formulas for derivatives and their errors;
- we will use Taylor polynomials and polynomial interpolants;
- we will use Richardson extrapolation to find formulas for higher order derivatives from lower order derivatives.

Our Problem

- we have a "black box" function $f(x)$: we can evaluate $f(x)$, but we do not have a closed form expression of $f(x)$;
- we seek to compute approximations of $f'(x)$, $f''(x)$, etc., at an arbitrary point $x = x_0$;
- we also want estimates or bounds on the error in the approximation;
- we also want numerically stable methods: minimal roundoff error.

Why Numerical Differentiation?

- it is a major tool in solving differential and integral equations numerically;
- approximating derivatives is ubiquitous in continuous optimization;
- the need to estimate derivatives from discrete data often arises in applications;
- there are others strategies for computing derivatives: automatic differentiation.

From the Definition of Derivative

The derivative of f at x_0 is

$$f'(x_0) = \lim_{h \to 0} \frac{f(x_0 + h) - f(x_0)}{h}.$$

So take

$$f'(x_0) \approx \frac{f(x_0 + h) - f(x_0)}{h}.$$

- the discretization or truncation error decreases as h shrinks, as long as f is smooth enough;
- f is sufficiently smooth: high derivatives of f exist and are bounded;
- curse of the round-off error due to subtraction;

Taylor Approximation

- the derivative is a local operator;
- hence Taylor polynomial approximations come naturally into play;

Forward Difference Rule

- consider the first-order Taylor series about x_0:

$$f(x) = f(x_0) + f'(x_0)(x - x_0) + \frac{1}{2}f''(\xi)(x - x_0)^2$$

- let $h = x - x_0 > 0$:

$$f(x_0 + h) = f(x_0) + f'(x_0)h + O(h^2)$$

- then, the **Forward Difference Rule** (scheme) is

$$f'(x_0) = \frac{1}{h}\left(f(x_0 + h) - f(x_0)\right) + O(h)$$

- the truncation error decreases linearly with h;
- the **Backward Difference Rule** is similar but with $h < 0$;
- the forward and backward difference rules are one-sided, two-point formulas with truncation error $O(h)$, i.e., they are 1st order methods.

Example: Forward Difference Rule

If $f(x) = \sin(x)$, what is $f'(1.2)$?

- we are approximating $\cos(1.2) = 0.362357754476674$;
- performing the computation with decreasing values of h, we get

h	Absolute error
0.1	4.716676e-2
0.01	4.666196e-3
0.001	4.660799e-4
1.e-4	4.660256e-5
1.e-7	4.619326e-8

- it looks the approximation error is behaving as expected: smaller h, smaller error.

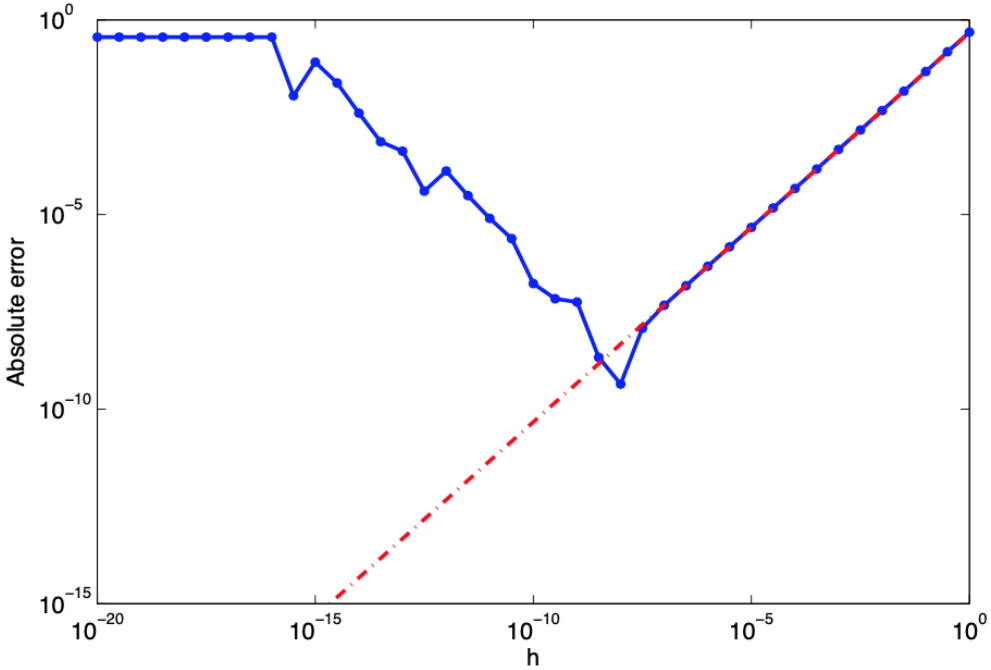
Smaller h

- continuing with smaller values of h:

h	Absolute error
1.e-8	4.361050e-10
1.e-9	5.594726e-8
1.e-10	1.669696e-7
1.e-11	7.938531e-6
1.e-13	6.851746e-4
1.e-15	8.173146e-2
1.e-16	3.623578e-1

- these results reflect the effect of both truncation error and roundoff errors;
- roundoff error *increases* with decreasing h.

Results

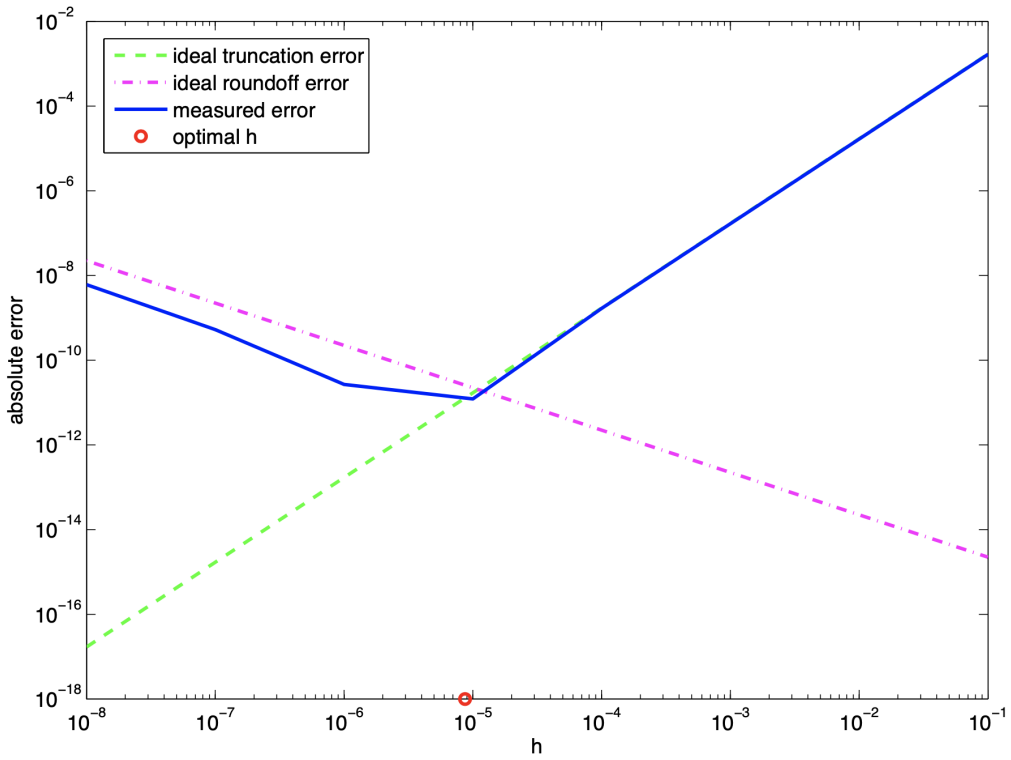


- solid curve: computed values for $|f'(x_0) - \frac{f(x_0+h)-f(x_0)}{h}|$;
- dashed line: theoretical truncation error (without roundoff error);
- dashed line: the slope is 1 on a log-log plot; why?

Roundoff Error

- for very small h and sufficiently smooth f, we may encounter severe cancellation error amplified by h^{-1} when approximating the derivative f';
- this is a common problem in all numerical differentiation methods;
- we would like to use a sufficiently large h so our results are not polluted by roundoff error;
- this is the justification for higher order methods $O(h^r)$ with $r > 1$: truncation error decreases faster and we achieve the desired accuracy before increased roundoff error.

Errors



- there is a "theoretically optimal" h, which is not always easy to determine;
- in practice, we tend to operate in the regime where roundoff error is smaller than truncation error;
- rule of thumb: $h \gg \eta^{1/(q+1)}$, q is the accuracy order of the method.

Example

Assuming a tolerance of $1/2 \times 10^{-6}$ for the truncation error, estimate when to stop decreasing h when calculating $f'(0.5)$ for $f(x) = \sin x$ using the forward difference rule with $h = 4^{-n}$.

Central Difference Rule

Can we manipulate the Taylor expansion to create a truncation error $O\left(h^q\right)$, with $q \geq 2$? Yes, we can:

$$f'(x) = \frac{f(x+h) - f(x-h)}{2h} - \frac{h^2}{6} f'''(\xi),$$

with ξ between $x - h$ and $x + h$.

Also known as the **Three-Point Midpoint Rule**.

Example

Assuming a tolerance of $1/2 \times 10^{-6}$ for the truncation error, estimate when to stop decreasing h when calculating $f'(0.5)$ for $f(x) = \sin x$ using the central difference rule with $h = 4^{-n}$.

Three-Point Endpoint Formula

We can manipulate the Taylor expansion to get the **three-point endpoint formula**:

$$f'(x) = \frac{1}{2h}\left[-3f(x) + 4f(x+h) - f(x+2h)\right] + \frac{h^2}{3}f'''(\xi_1),$$

with ξ_1 between x and $x + 2h$.

The formula has approximately a doubled truncation error comparing to the previous three-point formula because it uses information from one side of the function.

Five-Point Formulas

- **Five-Point Midpoint**

$$\begin{aligned} f'(x) &= \frac{1}{12h}\left[f(x-2h) - 8f(x-h) + 8f(x+h) - f(x+2h)\right] \\ &+ \frac{h^4}{30} f^{(5)}(\xi_2), \end{aligned}$$

with ξ_2 between $x-2h$ and $x+2h$.

- **Five-Point Endpoint**

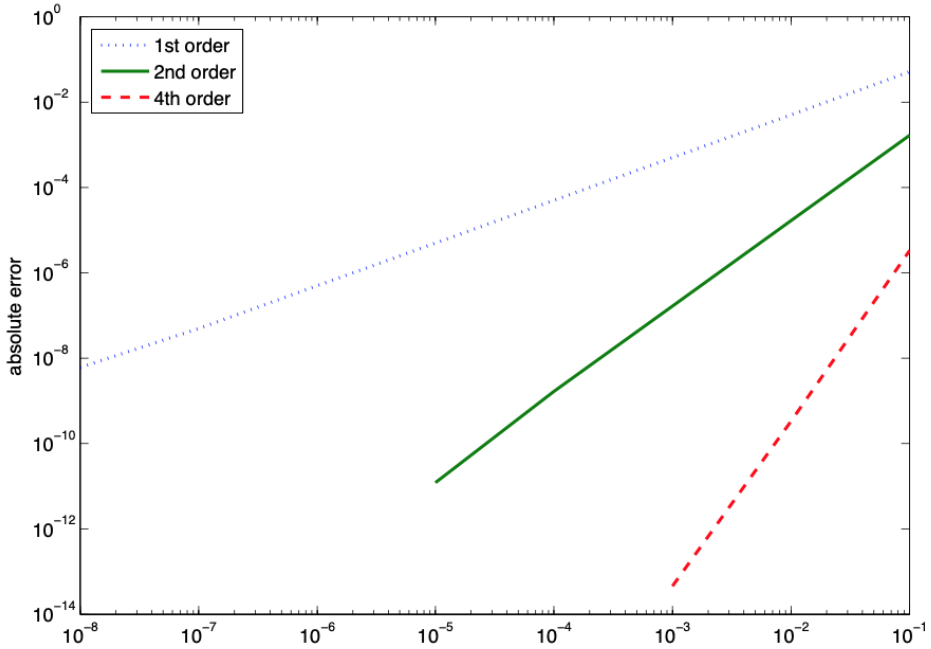
$$\begin{aligned} f'(x) &= \frac{1}{12h}\left[-25f(x) + 48f(x+h) - 36f(x+2h)\right. \\ &+ \left.16f(x+3h) - 3f(x+4h)\right] + \frac{h^4}{5} f^{(5)}(\xi_3). \end{aligned}$$

with ξ_3 between x and $x+4h$.

Example

- approximate $f'(0)$ for $f(x) = e^x$ using methods of order 1, 2, and 4:

h	$\frac{e^h-1}{h} - 1$	$\frac{e^h-e^{-h}}{2h} - 1$	$\frac{-e^{2h}+8e^h-8e^{-h}+e^{-2h}}{12h} - 1$
0.1	5.17e-2	1.67e-3	3.33e-6
0.01	5.02e-3	1.67e-5	3.33e-10
0.001	5.0e-4	1.67e-7	−4.54e-14
0.0001	5.0e-5	1.67e-9	−2.60e-13
0.00001	5.0e-6	1.21e-11	−3.63e-12



Central Finite Differences Matrix

$$f' \approx Df$$

$$\begin{bmatrix} f'(x_1) \\ f'(x_2) \\ \vdots \\ f'(x_{n-1}) \end{bmatrix} \approx \overbrace{\frac{1}{2h}\begin{bmatrix} -1 & 0 & 1 & & & \\ & -1 & 0 & 1 & & \\ & & \ddots & \ddots & \ddots & \\ & & & -1 & 0 & 1 \end{bmatrix}}^{D} \begin{bmatrix} f(x_0) \\ f(x_1) \\ f(x_2) \\ \vdots \\ f(x_{n-1}) \\ f(x_n) \end{bmatrix}$$

- the $(n-1) \times (n+1)$ matrix D is an instance of a *differentiation matrix*.
- D is not square, let alone invertible.